NumPy Exam Code Cheat Sheet

Essential syntax, array manipulations, indexing, vectorization & linear algebra

Topic / Module	Primary Functions & Syntax	Key Exam Concept
Array Creation	np.array(), np.zeros(), np.ones(), np.arange(), np.linspace()	Dimensionality, shapes, data types
Indexing & Slicing	arr[r, c], arr[arr > k], arr[[r1, r2]], arr[::-1]	Views vs copies, boolean masking
Reshaping & Stacking	arr.reshape(), arr.T, np.vstack(), np.hstack()	Memory layout, axis alignment
Broadcasting	arr 2D + arr 1D, scalar operations	Dimension compatibility rules
Aggregations	np.sum(), np.mean(), np.argmax(), axis=0 vs axis=1	Reducing along specific axes
Linear Algebra	np.dot(), @, np.linalg.inv(), np.linalg.det()	Matrix multiplication & inversion

1. Array Creation & Attribute Inspection

```
import numpy as np

# 1. Creating Arrays
arr_1d = np.array([10, 20, 30, 40])                 # 1D Array (Vector)
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])           # 2D Array (Matrix - 2x3)

# 2. Built-in Initializers
zeros  = np.zeros((3, 4), dtype=np.float64)          # 3x4 filled with 0.0
ones   = np.ones((2, 3), dtype=np.int32)             # 2x3 filled with 1
full   = np.full((2, 2), 7)                          # 2x2 filled with 7
identity = np.eye(3)                                 # 3x3 Identity Matrix

# 3. Numerical Ranges
seq_step = np.arange(0, 10, 2)                       # [0, 2, 4, 6, 8] (start, stop, step)
seq_line = np.linspace(0, 1, 5)                      # [0.  , 0.25, 0.5 , 0.75, 1.  ] (evenly spaced)

# 4. Attribute Inspection (CRITICAL FOR EXAMS)
print(arr_2d.shape)  # (2, 3)  -&gt; (rows, columns)
print(arr_2d.ndim)   # 2       -&gt; Number of dimensions (axes)
print(arr_2d.size)   # 6       -&gt; Total number of elements
print(arr_2d.dtype)  # int64   -&gt; Element data type
```

EXAM TIP: np.arange(start, stop, step) excludes 'stop', whereas np.linspace(start, stop, num) INCLUDES 'stop' by default and specifies the total count of points!

2. Indexing, Slicing & Boolean Masking

```
matrix = np.array([[10, 20, 30],
                   [40, 50, 60],
                   [70, 80, 90]])

# 1. Basic 2D Slicing: arr[row_slice, col_slice]
elem      = matrix[1, 2]         # 60 (row index 1, col index 2)
row_0     = matrix[0, :]         # [10, 20, 30] (First row)
col_1     = matrix[:, 1]         # [20, 50, 80] (Second column)
sub_grid  = matrix[0:2, 1:3]     # [[20, 30], [50, 60]] (2x2 sub-matrix)

# 2. Reversing Arrays
reversed_1d = arr_1d[::-1]       # Reverses 1D array

# 3. Boolean Indexing (Masking &amp;amp; Filtering)
mask = matrix &amp;gt; 40               # Returns boolean array of same shape
values_above_40 = matrix[mask]   # [50, 60, 70, 80, 90] (1D result)
matrix[matrix &amp;lt; 50] = 0         # In-place modification of matching elements

# 4. Fancy Indexing (Integer Arrays)
rows_to_pick = [0, 2]
extracted = matrix[rows_to_pick] # Selects row 0 and row 2
```

EXAM TIP: Standard slicing (e.g. arr[0:2]) creates a VIEW (modifying it changes the original array). Fancy indexing and Boolean masking return a COPY!

3. Reshaping, Transposing & Stacking

```
a = np.arange(12)                    # [0, 1, ..., 11] (1D)

# 1. Reshaping (Total elements must remain identical!)
grid = a.reshape(3, 4)               # Reshapes 12 elements into (3, 4)
auto_col = a.reshape(3, -1)          # -1 auto-calculates remaining dimension (4)

# 2. Flattening Arrays
flat_view = grid.ravel()             # 1D View (Shares memory)
flat_copy = grid.flatten()           # 1D Copy (Allocates new memory)

# 3. Transposing
transposed = grid.T                  # Swaps axes: shape becomes (4, 3)

# 4. Stacking &amp;amp; Concatenation
x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])

v_stack = np.vstack((x, y))          # Vertical stack -&amp;gt; shape (4, 2)
h_stack = np.hstack((x, y))          # Horizontal stack -&amp;gt; shape (2, 4)
c_axis0 = np.concatenate((x, y), axis=0) # Equivalent to vstack
c_axis1 = np.concatenate((x, y), axis=1) # Equivalent to hstack
```

EXAM TIP: Using -1 in reshape (e.g., arr.reshape(2, -1)) lets NumPy automatically calculate the dimension length based on array size.

4. Aggregations & Axis Operations

```
M = np.array([[1, 2, 3],
              [4, 5, 6]])

# 1. Global Aggregations
total_sum = np.sum(M)               # 21 (Sum of all elements)
mean_val  = np.mean(M)              # 3.5
std_val   = np.std(M)               # Standard deviation
min_val   = np.min(M)               # 1
max_val   = np.max(M)               # 6

# 2. Axis-Specific Operations (EXAM FAVORITE!)
# axis=0: Operation DOWN the columns (collapses rows)
col_sum = np.sum(M, axis=0)         # [5, 7, 9] (Shape: (3,))

# axis=1: Operation ACROSS the rows (collapses columns)
row_sum = np.sum(M, axis=1)         # [6, 15]   (Shape: (2,))

# 3. Finding Indices of Min/Max
max_index_flat = np.argmax(M)       # 5 (Index in flattened 1D array)
col_min_indices = np.argmin(M, axis=0) # [0, 0, 0] (Row index of min in each col)
```

EXAM TIP: Axis Rule: axis=0 operates DOWN columns (collapsing row dimension); axis=1 operates ACROSS rows (collapsing column dimension)!

5. Vectorization & Broadcasting Rules

```
# 1. Element-wise Vectorized Operations
u = np.array([1, 2, 3])
v = np.array([10, 20, 30])

add_res = u + v                     # [11, 22, 33]
mul_res = u * v                     # [10, 40, 90] (Element-wise, NOT matrix multiplication)
exp_res = np.exp(u)                 # e^x applied to every element
sqrt_res = np.sqrt(v)               # Square root applied element-wise

# 2. Broadcasting Rules
# Two dimensions are compatible if:
# Rule 1: They are equal in size, OR
# Rule 2: One of the dimensions is 1.

A = np.ones((3, 3))                 # Shape: (3, 3)
b = np.array([1, 2, 3])             # Shape: (3,) -&gt; treated as (1, 3)

# b is automatically stretched across rows to match (3, 3)
result = A + b
# [[2., 3., 4.],
#  [2., 3., 4.],
#  [2., 3., 4.]]
```

EXAM TIP: In NumPy, the operator * performs element-wise multiplication! For matrix multiplication, use @ or np.dot().

6. Linear Algebra & Pseudorandom Numbers

```
# 1. Linear Algebra Operations
X = np.array([[1, 2], [3, 4]])
Y = np.array([[2, 0], [1, 2]])

dot_prod = np.dot(X, Y)              # Matrix Multiplication
matmul_op = X @ Y                    # Preferred modern syntax for matrix mult
det_X     = np.linalg.det(X)         # Determinant (-2.0)
inv_X     = np.linalg.inv(X)         # Inverse Matrix
e_vals, e_vecs = np.linalg.eig(X)    # Eigenvalues and Eigenvectors

# 2. Random Number Generation
np.random.seed(42)                   # Sets seed for exact reproducibility

rand_unif = np.random.rand(2, 3)     # Uniform distribution [0, 1) shape (2, 3)
rand_norm = np.random.randn(2, 3)    # Standard Normal distribution (mean 0, std 1)
rand_ints = np.random.randint(1, 10, size=(2, 3)) # Random integers from [1, 10)
```

EXAM TIP: Always call np.random.seed(value) before generating random numbers when reproducible test or exam results are required!

7. Common Exam Pitfalls Summary

Operation	Syntax / Function	Common Mistake / Trap to Avoid
Matrix Multiply	A @ B or np.dot(A, B)	Using A * B (which does element-wise multiplication instead!)
Views vs Copies	b = a[0:2] vs b = a.copy()	Forgetting that slicing creates a view, so modifying b modifies a!
Axis Collapsing	np.sum(arr, axis=0)	Confusing axis=0 (operates vertically down columns) with axis=1 (rows).
Reshape Size	arr.reshape(rows, cols)	Specifying shape product that does not equal arr.size raises ValueError.
Boolean Indexing	arr[(arr > 2) & (arr < 8)]	Using 'and'/'or' instead of bitwise '&'/'|' with parentheses in NumPy.